

PROGRAMAÇÃO DE CLIENTES WEB

JavaScript

LINGUAGEM JAVASCRIPT

- Linguagem de programação interpretada que roda no Navegador do usuário (front-end)

Ao clicar em um elemento de site e desencadeia uma ação, isso é JavaScript

Melhora a experiência do usuário

- Existe também uma versão back-end

RECURSOS

- Criação de aplicações web, mobile (React Native), desktop

- Empresas famosas que usam

Facebook

Google

Uber

Netflix

Tiktok

...

EVOLUÇÃO

Linguagem em constante mudança



JS



JAVASCRIPT

- Tipos de Dados
- Variáveis
- Funções
- Estruturas condicionais e de repetição
- Características de linguagens funcionais

VAMOS TRABALHAR?

- No navegador
- No terminal (Node.js)

OPERADORES JAVASCRIPT

- atribuição

Operator	Example	Same As
=	x = y	x = y
+=	x += y	x = x + y
-=	x -= y	x = x - y
*=	x *= y	x = x * y
/=	x /= y	x = x / y
%=	x %= y	x = x % y
**=	x **= y	x = x ** y

OPERADORES JAVASCRIPT

- aritméticos

Operator	Description
+	Addition
-	Subtraction
*	Multiplication
**	Exponentiation (<u>ES2016</u>)
/	Division
%	Modulus (Division Remainder)
++	Increment
--	Decrement

OPERADORES JAVASCRIPT

- comparação

Operator	Description
==	equal to
===	equal value and equal type
!=	not equal
!==	not equal value or not equal type
>	greater than
<	less than
>=	greater than or equal to
<=	less than or equal to
?	ternary operator

OPERADORES JAVASCRIPT

- lógicos

Operator	Description
&&	logical and
	logical or
!	logical not

- de tipo

Operator	Description
typeof	Returns the type of a variable
instanceof	Returns true if an object is an instance of an object type

TIPOS DE DADOS

- String
- Number
- Boolean
- Undefined
- null
- Object
- Array
- Date
- Function (???)

STRING

- Cadeia de caracteres
 - “” - aspas duplas
 - ” - aspas simples
 - `` - template strings

STRING PROPRIEDADE DE TAMANHO

- Tamanho – length

```
let minhaString = "ABCDEF";
```

```
let tamanho = minhaString.length;
```

STRING – CHARACTER DE ESCAPE

`let text = “We are the so-called “Vikings” from the north.”;`

use contrabarra - \

\’ – resultado: ‘

\” – resultado: “

\\ - resultado: \

STRING – CHARACTER DE ESCAPE

```
let text = “We are the so-called \“Vikings\” from the north.”;
```

Código	Resultado
\b	backspace
\f	Form feed
\n	Nova Linha
\r	Carriage Return
\t	Tabulação
\v	Tabulação vertical

STRING MÉTODOS

- `slice(start, end)`
extraí uma parte de uma string e retorna a parte extraída em uma nova string
- `substring(start, end)`
similar ao `slice`, só que não aceita índices negativos
- `substr(start, length)`
similar ao `slice`, sendo o segundo parâmetro o tamanho da nova string

EXEMPLOS

```
let str = "Apple, Banana, Kiwi";  
let part = str.slice(-12, -6);
```

```
let str = "Apple, Banana, Kiwi";  
let part = str.substring(7, 13);
```

```
let str = "Apple, Banana, Kiwi";  
let part = str.substr(7, 6);
```

STRING – ALTERANDO CONTEÚDOS

- Replace - Substituindo conteúdos

```
let text = "Please visit Microsoft!";  
let newText = text.replace("Microsoft", "W3Schools");
```

- toUpperCase() – Convertendo em maiúsculo

```
let text1 = "Hello World!";  
let text2 = text1.toUpperCase();
```

- toLowerCase() – Convertendo em maiúsculo

```
let text1 = "Hello World!";    // String  
let text2 = text1.toLowerCase(); // text2 is text1 converted to lower
```

EXTRAINDO TRECHOS DE STRINGS

- `charAt(position)` – retorna o caracter na posição informada (0 a n-1)

```
let text = "HELLO WORLD";  
let char = text.charAt(0);
```

- `charCodeAt(position)` – retorna o código ASCII na posição informada (0 a n-1)

```
let text = "HELLO WORLD";  
let char = text.charCodeAt(0);
```

- `property access[]` – retorna o caracter na posição informada

```
let text = "HELLO WORLD";  
let char = text[0];
```

CONVERTENDO UMA STRING EM ARRAY

- `split()`

`text.split(",")` // Split on commas

`text.split(" ")` // Split on spaces

`text.split("|")` // Split on pipe

BUSCA EM UMA STRING

- `indexOf()` // aceita um segundo parâmetro definindo onde começar a buscar

```
let str = "Please locate where 'locate' occurs!";  
str.indexOf("locate");
```

- `lastIndexOf()`

```
let str = "Please locate where 'locate' occurs!";  
str.lastIndexOf("locate");
```

- `search()` // não aceita segundo parâmetro; aceita expressão regular

```
let str = "Please locate where 'locate' occurs!";  
str.search("locate");
```

- `startsWith()` e `endsWith()` // retorna verdadeiro/falso ; segundo parâmetro

```
let str = "Please locate where 'locate' occurs!";  
str.startsWith("locate");
```

STRING – REFERÊNCIA COMPLETA

https://www.w3schools.com/jsref/jsref_obj_string.asp

JS NUMBERS

- Javascript só tem um tipo de números – com ou sem casas decimais.
- Números em Javascript sempre tem 64bits de ponto flutuante
- O número máximo de casas decimais é 17
- A precisão dos pontos flutuantes não é muito confiável

JS NUMBERS

- **NaN** – Not a Number : palavra reservada que indica que um número está incorreto

```
let x = 100 / "Apple";
```

```
isNaN(x);
```

- **Infinity (ou -Infinity)** : valor que descreve que o resultado está acima da capacidade de representação

Math.abs

- Use a função abs para retornar o valor absoluto de um número (sem sinal)

```
console.log(Math.abs(5)) // Output é 5
```

```
console.log(Math.abs(5.2)) // Output é 5.2
```

```
console.log(Math.abs(-5.2)) // Output é 5.2
```

Math.ceil

- Use a função ceil para arredondar o valor de um número de ponto flutuante para cima

```
console.log(Math.ceil(0.8)) // Output é 1
```

```
console.log(Math.ceil(-0.8)) // Output é 0
```

```
console.log(Math.ceil(4)) // Output é 4
```

Math.floor

- Use a função floor para arredondar o valor de um número de ponto flutuante para o primeiro inteiro abaixo

```
console.log(Math.floor(0.8)) // Output é 0
```

```
console.log(Math.floor(-0.8)) // Output é -1
```

```
console.log(Math.floor(4)) // Output é 4
```

Math.round

- Use a função round para arredondar o valor de um número de ponto flutuante para o inteiro mais próximo

```
console.log(Math.round(0.8)) // Output é 1
```

```
console.log(Math.round(-0.8)) // Output é -1
```

```
console.log(Math.round(4)) // Output é 4
```

```
console.log(Math.round(4.95)) // Output é 5
```

```
console.log(Math.round(4.05)) // Output é 4
```

Math.trunc

- Use a função trunc corta as casas decimais de um número de ponto flutuante

```
console.log(Math.trunc(0.8)) // Output é 0
```

```
console.log(Math.trunc(-0.8)) // Output é 0
```

```
console.log(Math.trunc(4)) // Output é 4
```

```
console.log(Math.trunc(4.95)) // Output é 4
```

```
console.log(Math.trunc(4.05)) // Output é 4
```

Math.max

- Use a função max retorna o maior valor de uma coleção

`console.log(Math.max(2, 0.8, 5, -15))` // Output é 2

Math.min

- Use a função min retorna o menor valor de uma coleção

```
console.log(Math.min(2, 0.8, 5, -15)) // Output é -15
```

Math.pow

- Use a função pow retorna o potência: parâmetros – base e expoente

```
console.log(Math.pow(2, 5)) // Output é 32
```

Equivalente em EcmaScript 6:

```
console.log(2 ** 5) // Output é 32
```


Math.sqrt

- Use a função sqrt retorna a raiz quadrada

```
console.log(Math.sqrt(9)) // Output é 3
```

Equivalente em EcmaScript 6:

```
console.log(9 ** (1/2))) // Output é 3
```

Math.cbrt

- Use a função cbrt retorna a raiz cúbica

```
console.log(Math.cbrt(27)) // Output é 3
```

Equivalente em EcmaScript 6:

```
console.log(27 ** (1/3)) // Output é 3
```

Math.random

- Use a função random retorna um número pseudo-aleatório entre 0 e 1

`console.log(Math.random())` // Output é 0.973243223454322

NUMBER – REFERÊNCIA COMPLETA

https://www.w3schools.com/js/js_numbers.asp

BOOLEANS

VALORES – VERDADEIRO OU FALSO

- Criando um tipo booleano

```
let isentoDeProva = true
```

```
let aprovado = false
```

```
console.log(isentoDeProva.toString()); //  
"true"
```

```
console.log(aprovado.toString()) // "false"
```

CUIDADO! TRUTHY & FALSY

- JavaScript assume o estado de determinados tipos como **true** ou **false**

```
!!0; // false
```

```
!!NaN; //false
```

```
!!""; //false
```

```
!!false; //false
```

```
!!null; //false
```

```
!!undefined; //false
```

UNDEFINED

- O tipo **undefined** é retornado caso uma propriedade de um determinado objeto seja consultada e não exista

```
let mundialPalmeiras; // não há atribuição à variável  
console.log(mundialPalmeiras); // undefined
```

UNDEFINED

NULL

NULL

- Indica ausência de valor em uma determinada propriedade já existente.

```
let aluno = {};
```

```
aluno.p1 = null; // neste caso, a atribuição à p1 foi explicitamente declarada
```

```
console.log(aluno.p1); // null
```

OBJECT

OBJETO

- É uma **coleção dinâmica de chaves e valores** de qualquer tipo de dado
- É possível **adicionar** ou **remover** propriedades a qualquer momento

```
let pessoa = {  
  nome: "João",  
  idade: 20,  
  telefone: null,  
  endereco: {  
    logradouro: "Rua Brasil",  
    numero: 1301,  
    bairro: "Centro"  
  }  
}
```

```
pessoa.nome; // "João"  
pessoa["idade"]; // 20  
pessoa.telefone; // null  
pessoa.endereco.logradouro; //"Rua Brasil"  
pessoa["endereco"]["numero"]; // 1301  
pessoa.endereco.bairro; //"Centro"  
pessoa.peso; //undefined
```

REMOVENDO ATRIBUTOS DE OBJETOS

```
delete pessoa.telefone
```